samples from the probability distribution to take an action. Remember, an agent in reinforcement learning is whatever function or algorithm takes a state and returns a concrete action that will be executed in the environment.

Let's express this in code.

Listing 4.4. Setting up the policy network

```
import gym
import numpy as np
import torch

l1 = 4                          1
l2 = 150                        2
l3 = 2                          3

model = torch.nn.Sequential(
    torch.nn.Linear(l1, l2),
    torch.nn.LeakyReLU(),
    torch.nn.Linear(l2, l3),
    torch.nn.Softmax()          4
)

learning_rate = 0.0009
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

- ***1*** The input data is length 4 ("l1" is short for layer 1)
- ***2*** The middle layer produces a vector of length 150
- ***3*** The output is a 2-length vector for the left and right actions
- ***4*** The output is a softmax probability distribution over actions

That should all look fairly familiar to you at this point. The model is only two layers: a leaky ReLU activation function for the first layer, and the `Softmax` function for the last layer. We chose the leaky ReLU because it performed better empirically. You saw the `Softmax` function back in chapter 2; it just takes an array of numbers and squishes them into the range of 0 to 1 and makes sure they all sum to 1, basically creating a discrete probability distribution out of any list of numbers that are not probabilities to start with. For example, `softmax([-1,2,3]) = [0.0132, 0.2654, 0.7214]`. Unsurprisingly, the `Softmax` function will turn the bigger numbers into larger probabilities.